

• Quais são as entidades (classes) que devem ser criadas?

A estrutura deste cenário demanda a separação entre os atores do sistema, o documento regulador e o comportamento dinâmico dos serviços:

1. **Cliente (Entidade)**: Classe que armazena os dados de quem contrata o serviço, contendo nome e o endereço onde a equipe técnica deverá comparecer.
2. **Tecnico (Entidade)**: Classe correspondente ao profissional que executará a atividade, registrando seu nome e sua especialidade.
3. **OrdemServico (Entidade / Raiz de Agregação)**: A classe principal que dita o fluxo do processo. Ela une o cliente, o técnico e o serviço em um documento eletrônico, controlando datas, protocolos, o status e o preço de fechamento.
4. **ITipoServico (Interface/Abstração)**: Funciona como a entidade conceitual dos serviços. Em vez de criar propriedades estáticas, os tipos de serviço se tornam classes independentes (**ServicoManutencao**, **ServicoInstalacao**, **ServicoSuporte**) que compartilham essa mesma base.
5. **StatusOrdem (Enum)**: Enumerador que restringe as fases do atendimento: **Aberta**, **EmExecucao** e **Finalizada**.

• Quais dessas entidades possuem ações (métodos) e quais são eles?

Os comportamentos foram descentralizados para que cada objeto gerencie suas próprias regras com autonomia:

1. **Na classe OrdemServico:**
 - **AtribuirTecnico(Tecnico tecnico)**: Vincula um profissional disponível ao chamado e altera automaticamente o status do sistema para "Em Execução".
 - **ExecutarServico()**: Dispara o início dos trabalhos, acionando o comportamento técnico específico que foi contratado para aquele atendimento.
 - **FinalizarOrdem(int horasTrabalhadas)**: Altera o estado do chamado para "Finalizada", carimba o horário de encerramento e calcula o valor que o cliente deve pagar.
2. **Nas classes de Serviços (ServicoManutencao, ServicoInstalacao e ServicoSuporte):**
 - **Executar(string localCliente)**: Cada classe possui sua própria versão deste método, imprimindo ou processando rotinas físicas totalmente diferentes baseadas na sua natureza técnica.

• Há necessidade de criação de interface? Se sim, descreva os comportamentos que devem ser criados.

Sim, a criação da interface **ITipoServico** é o ponto central para resolver o requisito de que "cada serviço possui uma forma de execução e preço diferente". Em arquitetura de

software, esse padrão se chama *Strategy*. Ele evita o uso de condicionais aninhadas (*if/else*) e permite que novos serviços surjam no futuro sem que a classe de ordens de serviço precise ser modificada.

- *Comportamentos/Contratos*:
 - Propriedade `string Nome { get; }` para identificação.
 - Propriedade `decimal PrecoBase { get; }` para precificação individual.
 - Método `void Executar(string localCliente);` para isolar a forma como o trabalho é feito.

• Quais são os relacionamentos entre as classes e entidades criadas?

O ecossistema se conecta através de vínculos de associação e dependências polimórficas:

1. **Associação Simples (1 para 1)**: Uma `OrdemServico` aponta para exatamente um `Cliente` e para exatamente um `Tecnico`. Os atores existem de forma independente no banco de dados, mas são vinculados quando a ordem ganha vida.
2. **Associação Polimórfica por Interface**: A `OrdemServico` possui um atributo do tipo `ITipoServico`. Ela não sabe se o serviço é uma instalação ou uma manutenção; ela apenas interage com a interface abstrata, deixando que o motor do C# decida qual classe executar em tempo de execução.
3. **Uso de Enumerador**: A `OrdemServico` carrega a propriedade `StatusOrdem` para ditar o estado do ciclo de vida daquele atendimento.

• Onde o desenvolvedor deve tratar exceções em caso de críticas no processo?

As validações seguem as boas práticas de segurança divididas por contexto de erro:

1. **Na Camada de Domínio (Dentro da classe `OrdemServico`)**: O desenvolvedor deve incluir travas de segurança lançando `InvalidOperationException` para impedir que o fluxo lógico do negócio seja quebrado por mau uso do sistema. Exemplos:
 - Bloquear a troca ou alocação de um técnico caso a ordem de serviço já esteja com o status de "Finalizada".
 - Impedir a chamada do método `ExecutarServico` se nenhum profissional tiver sido atribuído previamente àquele chamado.
 - Bloquear a finalização de uma ordem que ainda consta como "Aberta" (sem passar pela fase de execução).
2. **Na Camada de Apresentação / Aplicação**: O desenvolvedor deve envolver as chamadas de finalização e faturamento em blocos `try-catch`. Caso o domínio rejeite a operação por alguma das regras citadas acima, o sistema intercepta o erro técnico, salva os detalhes em um arquivo de log local para análise interna e devolve um aviso limpo e compreensível na tela para a secretária ou atendente da empresa.